# AI Carbon Tracker Project

Full-Stack Data Engineering Documentation

## 1. The Conceptual Architecture

This project implements a real-time data pipeline. Data flows from a simulated hardware sensor (Emitter) to a web server (FastAPI), is stored in a relational database (PostgreSQL), and is finally analyzed via a dashboard.

- **Producer:** Python script simulating GPU power draw.
- **Ingestion:** FastAPI handling HTTP POST requests.
- **Storage:** PostgreSQL using a Star Schema (Fact and Dimension tables).
- **Analytics:** Aggregation logic to calculate Costs and CO2.

## 2. Database Setup (pgAdmin 4)

### A. Create the Database

1. Open pgAdmin 4.
2. Right-click **Databases** > **Create** > **Database...**
3. Name: `carbon_tracker_db`.

### B. Define the Schema (Star Schema)

We used two tables linked by a `hardware_id`.

```
-- 1. Dimension Table: Hardware Info
CREATE TABLE dim_hardware (
    hardware_id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    type VARCHAR(50) NOT NULL,
    tdp_watts INTEGER
);
```

```
-- 2. Fact Table: Energy Logs
CREATE TABLE fact_compute_logs (
    log_id SERIAL PRIMARY KEY,
    job_id VARCHAR(50),
    hardware_id INTEGER REFERENCES dim_hardware(hardware_id),
    power_draw_watts FLOAT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

### C. Seed Initial Data

You manually inserted the "NVIDIA A100" so the logs had something to join against:

```
INSERT INTO dim_hardware (name, type, tdp_watts) VALUES ('NVIDIA A100', 'GPU', 4
```

## 3. Application Setup

### A. Project Structure

Organizing code into an `app/` package for professional modularity:

```
carbon_tracker_project/
├── app/
│   ├── routes/tracking.py
│   ├── database.py
│   ├── models.py
│   ├── schemas.py
│   ├── main.py
│   └── dashboard.py
├── .env
├── .gitignore
└── test_emitter.py
```

### B. Security & Environment

We moved the `DATABASE_URL` to a `.env` file to keep the password safe from GitHub.

```
DATABASE_URL=postgresql+asyncpg://postgres:PASSWORD@localhost:5432/carbon_tracke
```

## 4. The Logic (Python Implementation)

### Models (SQLAlchemy)

Mapping database tables to Python classes. We used `Mapped` and `mapped_column` for modern type-hinting.

### The Analytics Calculation

The dashboard performs the following math:

$$\textit{Total kWh} = (\ \Sigma\ \textit{Power Watts} \times \textit{1 second}\ ) / 3600 / 1000$$

$$\textit{Cost} = \textit{Total kWh} \times \$0.16$$

$$\textit{CO2} = \textit{Total kWh} \times \textit{0.4 kg}$$

## 5. Deployment & Version Control

### A. Git Initialization

We initialized Git and used a `.gitignore` to hide `.env` and `__pycache__`.

### B. GitHub Push

Commands used to upload the code:

```
git add .
git commit -m "Initial commit"
git push origin main
```

## Summary of Learned Skills

| Concept | Technology |
| --- | --- |
| Asynchronous Programming | Python `asyncio` |
| REST API Ingestion | FastAPI |
| Relational DB Joins | SQL / SQLAlchemy |
| Secret Management | Environment Variables (.env) |